Python Programming Fundamentals

Input and Output

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1. The input() Function

`input()` reads a line of text from the user and always returns a **string**.

```
name = input("What is your name? ")
print(f"Hello, {name}!")

# The return value is ALWAYS a string
age_str = input("How old are you? ")
print(type(age_str))    # <class 'str'>
print(age_str + 1)      # TypeError! Can't add str and int
```

1. The input() Function

`input()` reads a line of text from the user and always returns a **string**.

```
name = input("What is your name? ")
print(f"Hello, {name}!")

# The return value is ALWAYS a string
age_str = input("How old are you? ")
print(type(age_str))    # <class 'str'>
print(age_str + 1)      # TypeError! Can't add str and int
```

Key point: even if the user types `25`, Python stores it as the string `"25"`, not the integer `25`.

You must **convert** it if you want to do arithmetic.

Outline

2. Type Conversion

Convert the string returned by `input()` to the type you need.

```
# int() — convert to integer
age = int(input("Age: "))
next_year = age + 1
print(f"Next year you'll be {next_year}")

# float() — convert to decimal
price = float(input("Price: "))
with_tax = price * 1.23
print(f"With 23% VAT: €{with_tax:.2f}")

# str() — convert to string (rarely needed with input)
number = 42
message = "The answer is " + str(number)
```

2. Type Conversion

```
# bool() — almost never used with input — use comparison instead
response = input("Continue? (Y/N): ")
going = response.upper() == "Y"
```

Outline

3. Handling Bad Input with try/except

If the user types something that can't be converted, Python raises an exception.

```
# This crashes if user types "hello" instead of a number
age = int(input("Age: "))   # ValueError if not a number
```

3. Handling Bad Input with try/except

3. Handling Bad Input with try/except

If the user types something that can't be converted, Python raises an exception.

```
# This crashes if user types "hello" instead of a number
age = int(input("Age: "))   # ValueError if not a number
```

Safe version with try/except:

```
while True:
    try:
        age = int(input("Enter your age: "))
        if age < 0 or age > 120:
            print("Please enter a realistic age")
            continue
        break
    except ValueError:
        print("That's not a valid number. Please try again.")
```

3. Handling Bad Input with try/except

```
print(f"You are {age} years old")
```

Outline

4. Input Validation Pattern

The clean, reusable pattern for validated input:

```
def get_integer(prompt, minimum=None, maximum=None):
    """Prompt user until they enter a valid integer in range."""
    while True:
        try:
            value = int(input(prompt))
            if minimum is not None and value < minimum:
                print(f"Please enter at least {minimum}")
                continue
            if maximum is not None and value > maximum:
                print(f"Please enter at most {maximum}")
                continue
            return value
        except ValueError:
            print("Please enter a whole number")
```

4. Input Validation Pattern

```
age   = get_integer("Age: ", 0, 120)
score = get_integer("Score (0-100): ", 0, 100)
```

Outline

5. The print() Function

`print()` supports several useful options:

```
# Basic print
print("Hello, World!")

# Multiple values — separated by space by default
print("Name:", "Alice", "Age:", 25)
# Name: Alice Age: 25

# Custom separator
print("Alice", "Bob", "Carol", sep=", ")
# Alice, Bob, Carol

print(1, 2, 3, 4, 5, sep=" | ")
# 1 | 2 | 3 | 4 | 5
```

5. The print() Function

```
# Custom end (default is newline \n)
print("Loading", end="")
print(".", end="")
print(".", end="")
print(". Done!")
# Loading... Done!
```

Outline

6. print() for Debugging

```
# Check variable values
x = 42
print(f"{x = }")    # x = 42  (Python 3.8+ f-string debug syntax)

# Check multiple variables
a, b, c = 10, 20, 30
print(f"{a = }, {b = }, {c = }")   # a = 10, b = 20, c = 30

# Print with type info
value = 3.14
print(f"value = {value!r} (type: {type(value).__name__})")
# value = 3.14 (type: float)
```

6. print() for Debugging

```
# Check variable values
x = 42
print(f"{x = }")    # x = 42  (Python 3.8+ f-string debug syntax)

# Check multiple variables
a, b, c = 10, 20, 30
print(f"{a = }, {b = }, {c = }")   # a = 10, b = 20, c = 30

# Print with type info
value = 3.14
print(f"value = {value!r} (type: {type(value).__name__})")
# value = 3.14 (type: float)

# Tracing a loop
for i in range(5):
    print(f"Loop iteration {i}: value = {i**2}")
```

Outline

7. Practical — Interactive Calculator

```
def get_number(prompt):
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Please enter a number")

def get_operator():
    while True:
        op = input("Operator (+, -, *, /): ").strip()
        if op in "+-*/":
            return op
        print("Please enter +, -, * or /")

a = get_number("First number: ")
op = get_operator()
```

7. Practical — Interactive Calculator

```
b = get_number("Second number: ")

if op == "+":  result = a + b
elif op == "-": result = a - b
elif op == "*": result = a * b
elif op == "/":
    if b == 0:
        print("Cannot divide by zero")
    else:
        result = a / b
        print(f"{a} {op} {b} = {result:.4g}")
```

Thanks for Watching - Any questions?